

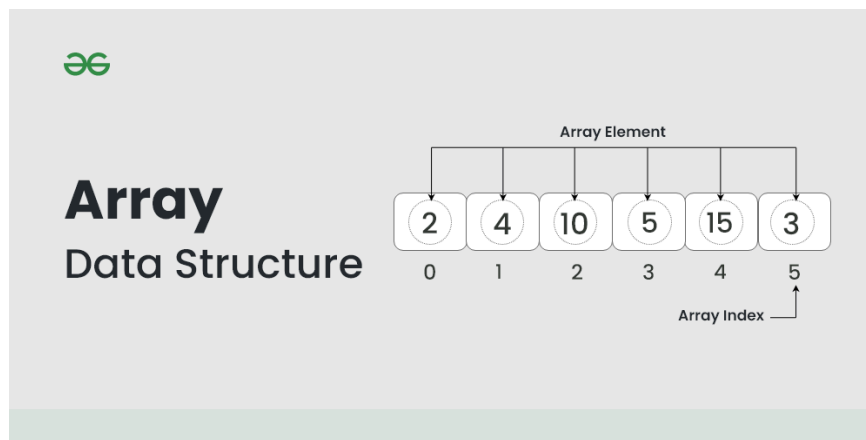
Aluna: Kauanny Ribeiro da Silva
Estrutura de Dados

RESUMO DO CONTEÚDO ATIVIDADE 26/11/2025

* *Arrays, Matrizes, Listas, Pilhas, Filas, Mapas e Tabelas Hash, Árvores*
* *Algoritmos de Ordenação e Busca*

Arrays

Uma coleção de elementos do mesmo tipo, armazenados em posições sequenciais de memória. Possuem tamanho fixo e permitem acesso rápido a qualquer elemento através de um índice. São a base para muitas outras estruturas de dados.



Matrizes

Arrays com múltiplas dimensões (como linhas e colunas). São úteis para representar dados tabulares, planilhas ou grades bidimensionais. Cada elemento é acessado através de múltiplos índices.

Boas Transmissões

Matrizes

Abaixo a matriz com alguns dados inseridos:

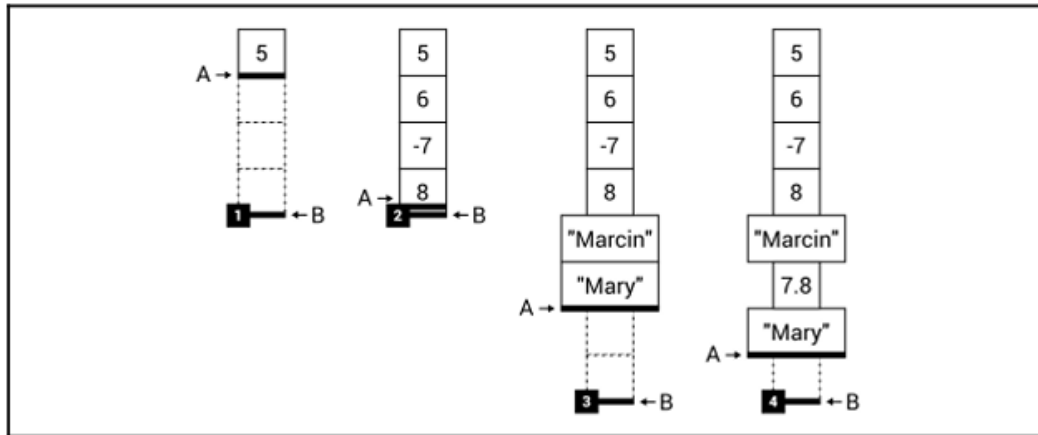
	0	1	2	3
0	22			
1			32	
2		15		
3			96	

Valor 22: posição 0,0 Valor 15: posição 2,1
Valor 32: posição 1,2 Valor 96: posição 3,2

Fábio dos Reis

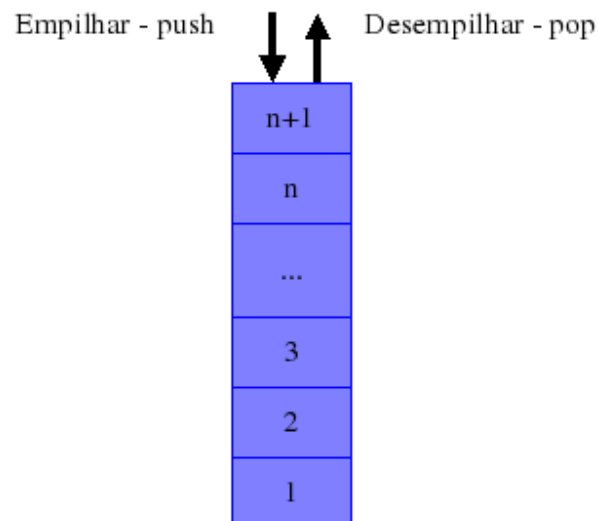
Listas

Coleções dinâmicas que podem crescer ou diminuir de tamanho conforme necessário. Diferente dos arrays, não possuem tamanho fixo e oferecem métodos convenientes para adicionar, remover e pesquisar elementos.



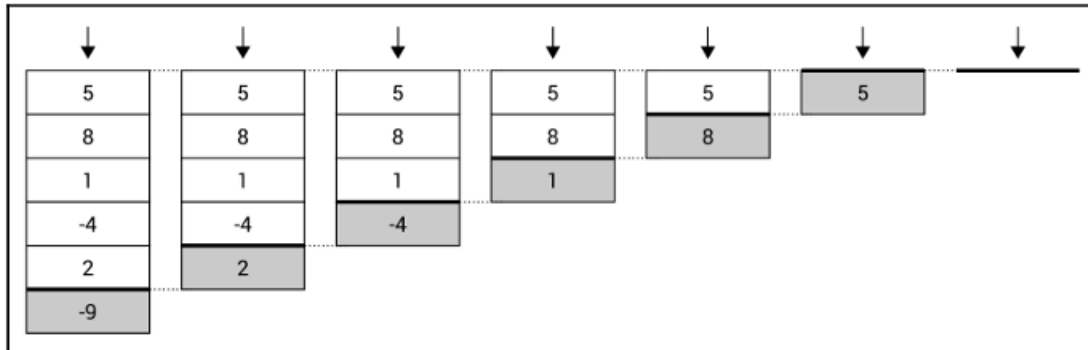
Pilhas

Estruturas que seguem o princípio LIFO (Last-In, First-Out) - o último elemento a entrar é o primeiro a sair. Operações básicas incluem Push (empilhar) e Pop (desempilhar). São usadas em cenários como histórico de navegação e avaliação de expressões.



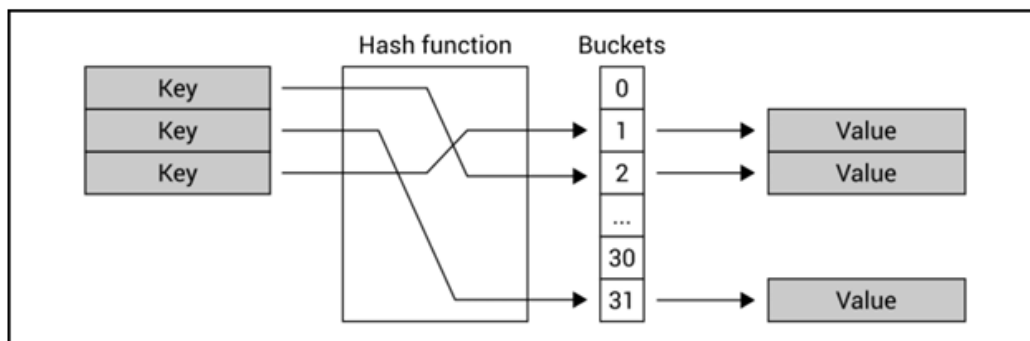
Filas

Estruturas que seguem o princípio FIFO (First-In, First-Out) - o primeiro elemento a entrar é o primeiro a sair. Operações básicas incluem Enqueue (enfileirar) e Dequeue (desenfileirar). Comuns em sistemas de atendimento e processamento em ordem.



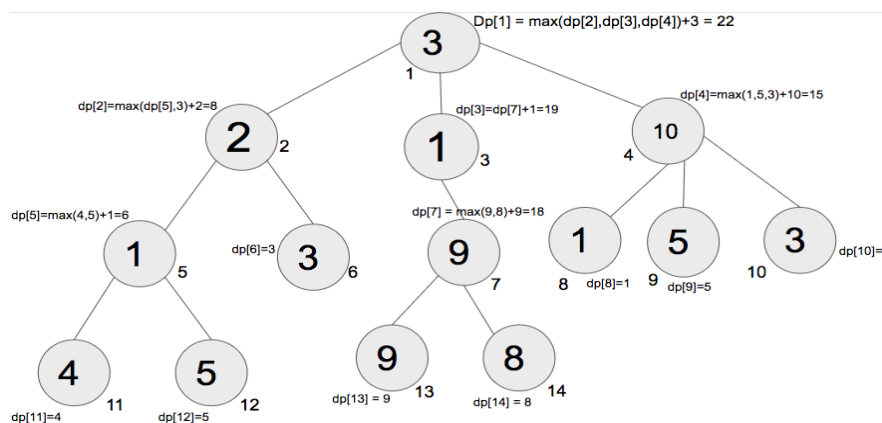
Mapas e Tabelas Hash

Coleções que armazenam pares chave-valor, permitindo busca extremamente eficiente. Usam funções hash para mapear chaves para posições específicas, proporcionando acesso rápido aos valores. Ideais para quando se precisa de consultas rápidas baseadas em chaves únicas.



Árvores

Estruturas hierárquicas não-lineares. As árvores binárias de busca mantêm os elementos ordenados, permitindo buscas, inserções e remoções eficientes. Árvores balanceadas (como AVL e Rubro-Negras) garantem que essas operações mantenham alta performance mesmo nos piores casos.



Algoritmos de Ordenação

Bubble Sort: Compara e troca elementos adjacentes repetidamente. Simples mas ineficiente para grandes conjuntos.

Funcionamento: Percorre repetidamente o array, comparando cada elemento com o próximo e trocando-os se estiverem na ordem errada. A cada passagem completa, o maior elemento "flutua" para sua posição correta no final do array, como uma bolha subindo.

Características:

Complexidade: $O(n^2)$ no pior e médio casos, $O(n)$ no melhor caso (quando já está ordenado)

Estável: não altera a ordem relativa de elementos iguais

In-place: usa apenas uma quantidade constante de memória extra

Ideal para: conjuntos pequenos ou dados quase ordenados

Processo:

Compara posição 0 com 1, troca se necessário

Avança comparando posições 1-2, 2-3, etc.

Repete até não haver mais trocas em uma passagem completa

Quick Sort: Usa a estratégia "dividir para conquistar", selecionando um pivô e particionando os dados. Muito eficiente na prática.

Funcionamento: Segue a estratégia "dividir para conquistar":

Escolhe um pivô (geralmente o primeiro, último ou elemento do meio)

Particiona o array: reorganiza os elementos menores que o pivô à esquerda e maiores à direita

Aplica recursivamente o mesmo processo às duas partições

Características:

Complexidade: $O(n \log n)$ no médio caso, $O(n^2)$ no pior caso (pivô mal escolhido)

Não é estável na implementação clássica

In-place: opera diretamente no array original

Mais rápido na prática para a maioria dos casos reais

Vantagens:

Cache-friendly: opera principalmente em posições adjacentes de memória

Baixa constante oculta nas operações

Merge Sort: Divide recursivamente a lista e depois combina as partes ordenadas. Eficiente e estável.

Funcionamento: Também usa "dividir para conquistar" de forma diferente:
Divide recursivamente o array até ter subarrays de 1 elemento (já ordenados)
Combina (merge) os subarrays ordenados produzindo arrays maiores ordenados

Características:

Complexidade: $O(n \log n)$ em todos os casos (pior, médio e melhor)
Estável: mantém a ordem original de elementos iguais
Não é in-place: requer espaço adicional $O(n)$ para o merging
Performance consistente independente do input

Aplicações:

Ordenação de listas encadeadas
Dados externos (que não cabem na memória principal)
Quando estabilidade é requerida

Heap Sort: Constrói uma estrutura heap para ordenar os elementos. Eficiente e com bom uso de memória.

Funcionamento:

Constrói um heap máximo a partir do array desordenado
Extraí repetidamente o elemento máximo (raiz do heap) e reconstrói o heap
Coloca o elemento máximo no final do array (parte já ordenada)

Características:

Complexidade: $O(n \log n)$ em todos os casos
Não é estável
In-place: usa o próprio array para construir o heap
Uso de memória: $O(1)$ - muito eficiente em memória

Vantagens:

Performance garantida $O(n \log n)$ sem casos patológicos
Excelente para sistemas com restrição de memória
Útil quando precisamos dos maiores/menores elementos

Algoritmos de Busca

Busca Linear: Percorre cada elemento sequencialmente até encontrar o desejado. Funciona em qualquer lista mas é lento para grandes volumes.

Funcionamento: Percorre cada elemento do início ao fim, comparando com o valor procurado até encontrá-lo ou chegar ao final.

Características:

Complexidade: $O(n)$ no pior caso

Não requer dados ordenados

Implementação simples e direta

Último recurso quando os dados não podem ser organizados

Casos de uso:

Listas muito pequenas

Buscas únicas em coleções não ordenadas

Quando a ordenação seria mais custosa que a busca em si

Busca Binária: Divide repetidamente uma lista ordenada pela metade até encontrar o elemento. Extremamente eficiente mas requer dados ordenados.

Funcionamento:

Assume uma coleção ordenada

Compara o elemento do meio com o valor buscado

Se for igual, retorna a posição

Se for maior, busca na metade inferior; se for menor, busca na metade superior

Repete até encontrar ou esgotar o espaço de busca

Características:

Complexidade: $O(\log n)$ - extremamente eficiente

Requer acesso aleatório aos elementos (arrays, não listas encadeadas)

Dados devem estar previamente ordenados

Variações:

Busca binária recursiva vs iterativa

Encontrar primeiro/última ocorrência em arrays com duplicatas

Busca em arrays rotacionados

Busca em Árvores: Aproveita a estrutura hierárquica das árvores binárias de busca para encontrar elementos de forma otimizada.

Funcionamento: Aproveita a propriedade das árvores binárias de busca onde:

Todos os elementos à esquerda de um nó são menores

Todos os elementos à direita são maiores

Processo:

Começa pela raiz

Compara com o valor alvo

Se for menor, vai para a subárvore esquerda; se for maior, vai para a direita

Repete até encontrar o nó ou chegar a uma folha nula

Características:

Complexidade: $O(h)$ onde h é a altura da árvore

Em árvores balanceadas: $O(\log n)$

Em árvores degeneradas (lista encadeada): $O(n)$

Vantagens:

Eficiente para dados dinâmicos (muitas inserções/remoções)

Não requer reorganização completa como a busca binária em arrays

Permite operações complexas como range queries

Tipos de Árvores para Busca:

Árvore Binária de Busca Simples

AVL Tree (balanceamento automático)

Red-Black Tree (balanceamento eficiente)

B-Tree (para sistemas de arquivos e bancos de dados)